

AU/NZ TRACTOR COMPARISON TOOL

Technical Implementation Specification

Version 1.1



Version 1.1 update

This revision confirms that every display-relevant field in machines.csv must be supported and adds derived power-to-weight comparison using Max HP and clean unladen weight values. The ratio is computed by the build/application layer, not stored as manually maintained CSV data.

Item	Decision
Product owner	Archie Ventrella
Primary development environment	Visual Studio Code with GitHub Copilot
Hosting	GitHub Pages
Source data	data-source/machines.csv
Implementation language	HTML, CSS, modern JavaScript and Node.js build scripts
Data scope	All display-relevant CSV specifications, plus helper, source and control fields
Version	1.1 10 September 2026

1. Implementation overview

The product will be implemented as a static web application. Contributors will maintain one CSV file, and an automated Node.js pipeline will validate and transform the CSV into complete JSON records consumed by the website. The front end will preserve the current relationship table, machine filters, detailed side-by-side comparison, safe numeric deltas, copy, CSV export and print functions.

Primary engineering principle

Application code changes infrequently. Machine data changes frequently. All display specifications must flow from the CSV through the generated JSON without being reduced to a headline subset.

1.1 Objectives

- Use machines.csv as the single source of truth for all machine, specification, filter, source, image and publication data.
- Support every display-relevant specification column in the detailed comparison view.
- Keep helper fields available for filters without duplicating them as customer-facing specification rows.
- Calculate derived power-to-weight values consistently without adding manually maintained ratio columns to the CSV.
- Derive manufacturers, model years and filter options from published CSV records.
- Block deployments where data errors could break the product or produce ambiguous machine identity.
- Keep all paths compatible with a GitHub Pages project URL.
- Provide a controlled implementation context for GitHub Copilot and future maintainers.

1.2 Technical boundaries

- No runtime database or server-side API.
- No browser-based data editing.
- No genuine user authentication in version 1.
- No secrets in client files or repository history.
- No hard-coded machine, make, year or filter catalogues.
- No manually entered power-to-weight ratio in the CSV.

2. Target architecture

2.1 Build-time and runtime flow

BUILD TIME
Contributor -> data-source/machines.csv -> validation -> transformation -> derived metrics -> dist/data/*.json -> GitHub Actions -> GitHub Pages

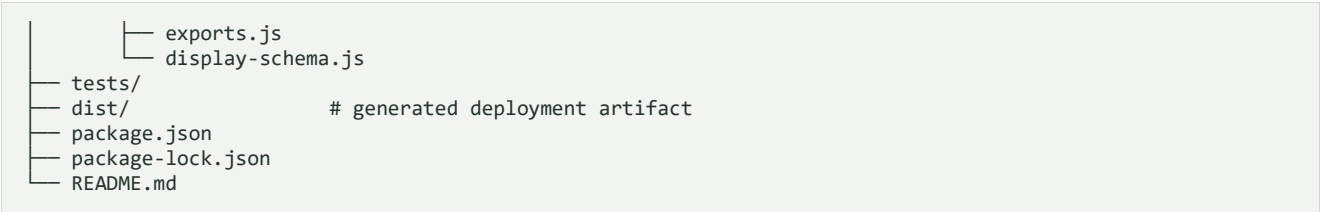
RUNTIME
Browser -> HTML + CSS + JavaScript -> fetch generated JSON -> render filters, relationships, power-to-weight and detailed comparisons

2.2 Technology decisions

Layer	Technology	Purpose
Development	Visual Studio Code	Local editing, terminal, Git source control and GitHub Copilot.
Source control	GitHub	Repositories, versioning, branches, pull requests and Pages.
Hosting	GitHub Pages	Static hosting for HTML, CSS, JS, JSON and images.
Build runtime	Node.js LTS	Cross-platform validation, transformation, tests and assembly.
Front end	Vanilla HTML, CSS and ES modules	Lightweight implementation aligned with the current browser tool.
Source data	UTF-8 CSV	Single human-maintained machine catalogue.
Runtime data	JSON	Complete structured machine records with derived metrics.
Automation	GitHub Actions	Validate, test, build and deploy reviewed changes.

2.3 Repository structure

```
aunz-tractor-comparison-tool/  
├── .github/  
│   ├── copilot-instructions.md  
│   ├── pull_request_template.md  
│   └── workflows/build-and-deploy.yml  
├── assets/images/machines/<manufacturer-slug>/  
├── data-source/machines.csv  
├── docs/  
│   ├── project-documentation.docx  
│   └── technical-implementation.docx  
├── scripts/  
│   ├── schema.js  
│   ├── csv-utils.js  
│   ├── derived-metrics.js  
│   ├── validate-data.js  
│   ├── build-data.js  
│   └── build-site.js  
├── src/  
│   ├── index.html  
│   ├── css/styles.css  
│   └── js/  
│       ├── app.js  
│       ├── data-loader.js  
│       ├── filters.js  
│       ├── relationships.js  
│       ├── comparison.js  
│       └── power-to-weight.js
```



3. Complete CSV data contract

The CSV contains the complete data set the product must support. Every CSV column must be represented in the schema. Display fields must be preserved in generated JSON and available in the detailed comparison. Helper fields drive filters. Control fields drive publication and identity. Source and maintenance fields support review and traceability.

3.1 Field groups and usage

Group	CSV fields	Product usage
Control and identity	machine_id, published, lifecycle_status, market	Identity, publication and market handling; not ordinary spec rows.
Image and source	image_filename, image_alt_text, source_url, source_title, last_reviewed_date, notes	Image display and optional source/review section.
Machine identity	machine, manufacturer, country_of_manufacture, model_year	Selection, filtering and comparison identity.
Power	rated_hp, max_hp, max_hp_with_ipm, max_torque_nm	Detailed comparison, relationship logic and derived metrics.
Transmission helpers	transmission, transmission_filter_tags, transmission_class	Transmission displays; helper values drive filters.
Speed and dimensions	top_speed_filter, cylinders_filter, top_speed_kmh, wheelbase_mm	Detailed display and filters where applicable.
Weights	unladen_weight_kg, max_permissible_weight_40_kmh_kg, max_permissible_weight_50_kmh_kg, max_permissible_weight_60_kmh_kg	Detailed comparison; unladen weight supports power-to-weight only when scalar.
Hydraulics and hitch	rated_hydraulic_flow_lpm, max_hydraulic_flow_lpm, max_scvs, rear_hitch_capacity_kg, front_hitch_capacity_kg	Detailed comparison; preserve multi-option wording.
Engine and cab	engine_capacity_l, number_of_cylinders, engine_manufacturer, cab_suspension	Detailed comparison.
Capacities and emissions	fuel_capacity_l, engine_stage_tier, adblue_tank_l	Detailed comparison; preserve source wording.
PTO	rear_pto_option, rear_pto_filter_tags	Rear PTO display and helper filtering.

3.2 Record grain and identity

- One row represents one machine/configuration, one model year and one market.
- A new model year is a new row.
- Previous valid model-year records remain available unless deliberately unpublished.
- machine_id is permanent, unique and composed of lowercase ASCII letters, numbers and hyphens.
- Do not reintroduce redundant brand abbreviations such as MF in Massey Ferguson IDs or display names.

3.3 Data preservation rules

- Blank, NA, N/A, No Data and ~ become JSON null.
- Clean numeric scalar columns may become JSON numbers.
- Pipe-separated helper fields become arrays.
- Descriptive values with options, qualifiers, ranges or configuration text remain complete strings.
- No build script or AI assistant may infer a missing tractor specification.
- Generated JSON is never the manual source of a correction.

3.4 Complete generated machine object

```
{
  "id": "john-deere-8r-410-2025-au",
  "published": true,
  "lifecycleStatus": null,
  "market": "AU",
  "machine": "John Deere 8R 410",
  "manufacturer": "John Deere",
  "modelYear": 2025,
  "specifications": {
    "countryOfManufacture": "USA",
    "ratedHp": 410,
    "maxHp": 443,
    "maxHpWithIpm": 445,
    "maxTorqueNm": 1851,
    "transmission": "eAutoPowr™ (EVT)",
    "topSpeedKmh": 50,
    "wheelbaseMm": 3050,
    "unladenWeightKg": 12700,
    "maxPermissibleWeight40KmhKg": 20003,
    "maxPermissibleWeight50KmhKg": 20003,
    "maxPermissibleWeight60KmhKg": null,
    "ratedHydraulicFlowLpm": "227 / 318",
    "maxHydraulicFlowLpm": 318,
    "maxScvs": "6 + 2",
    "rearHitchCapacityKg": "6894, 9072",
    "frontHitchCapacityKg": 5200,
    "engineCapacityL": 9,
    "numberOfCylinders": 6,
    "engineManufacturer": "John Deere",
    "cabSuspension": "Optional Adjustable Cab Suspension",
    "fuelCapacityL": 727,
    "engineStageTier": "Tier 3 / Final Tier 4",
    "adblueTankL": 37,
    "rearPtoOption": "540, 1000E, 1000"
  },
  "derived": {
    "powerToWeightHpPerTonne": 34.88,
    "powerToWeightKwPerTonne": 26.01,
    "powerBasis": "maxHp",
    "weightBasis": "unladenWeightKg"
  },
  "filters": { ... },
  "image": null,
  "source": { ... }
}
```

4. Derived power-to-weight implementation

Decision

Power-to-weight is a derived metric. It will not be manually entered in machines.csv. It will be calculated consistently by the build layer and included in generated JSON for direct display and comparison. This keeps the CSV factual and avoids stale or inconsistent ratios.

4.1 Initial business definition

Element	Version 1 definition
Power basis	max_hp
Weight basis	unladen_weight_kg
Primary display unit	hp per tonne
Secondary display unit	kW per tonne
Precision	Two decimal places for display; unrounded value may be retained internally.
Availability	Only when both source fields are valid positive scalar numbers.
CSV storage	Not stored.
Generated JSON	Stored under derived with basis metadata.

4.2 Formula

```
powerToWeightHpPerTonne = (maxHp / unladenWeightKg) * 1000
powerToWeightKwPerTonne = powerToWeightHpPerTonne * 0.745699872
```

Example using John Deere 8R 410:

```
maxHp = 443 hp
unladenWeightKg = 12,700 kg

hp/t = (443 / 12,700) * 1000 = 34.88 hp/t
kW/t = 34.88 * 0.745699872 = 26.01 kW/t
```

4.3 Eligibility rules

- max_hp must parse as a finite number greater than zero.
- unladen_weight_kg must parse as one finite scalar number greater than zero.
- Values such as 24639 (narrow), 25546 (wide) are descriptive, not scalar, and must not produce a ratio.
- Values such as 8000 / 8900 are configuration choices and must not produce a ratio unless a future explicit weight-basis field is introduced.
- Missing or ineligible inputs yield null ratios and a machine-readable unavailable reason.
- The calculation must never silently choose the first, minimum, maximum or average weight from a descriptive field.

4.4 Generated derived object

```
"derived": {
  "powerToWeightHpPerTonne": 34.88,
  "powerToWeightKwPerTonne": 26.01,
  "powerBasis": "maxHp",
  "weightBasis": "unladenWeightKg",
  "powerToWeightAvailable": true,
  "powerToWeightUnavailableReason": null
}
```

```
"derived": {
  "powerToWeightHpPerTonne": null,
  "powerToWeightKwPerTonne": null,
  "powerBasis": "maxHp",
  "weightBasis": "unladenWeightKg",
  "powerToWeightAvailable": false,
  "powerToWeightUnavailableReason": "unladen_weight_not_scalar"
}
```

4.5 User-interface behaviour

- Show Power-to-weight under a derived performance section in the detailed comparison.
- Display hp/t as the primary metric and kW/t as the secondary metric.
- Display the selected Max HP and unladen weight basis beside or beneath the ratio.
- Show a dash and concise explanation when the ratio is unavailable.
- Calculate the side-by-side absolute and percentage delta only when both ratios exist.
- Do not imply that the ratio represents loaded, ballasted, operating or maximum permissible weight performance.
- Include the ratio in copy, CSV export and print output when available.

4.6 Future extension

If the product later needs alternative bases, add an explicit selector and derived calculations for rated HP, maximum permissible weight, or other verified scalar weight bases. Do not overload the initial metric or infer configuration weights from descriptive values.

5. Validation and build pipeline

5.1 Schema requirement

scripts/schema.js must define all CSV columns, their source type, output location, required status, controlled values and whether the field is displayable, filter-only, control-only or source metadata. Validation and building must import the same schema.

5.2 Commands

```
npm run validate    # validate source CSV
npm run build       # validate, generate complete JSON and assemble dist
npm test           # run unit and integration tests
npm run dev         # build and serve dist locally
npm run clean       # remove generated output
```

5.3 Validation sequence

1. Verify the exact expected CSV header and all schema mappings.
2. Track source row number for every error and warning.
3. Validate required identity and publication fields.
4. Validate machine_id format and uniqueness.
5. Validate clean numeric fields only where configured as numeric.
6. Validate pipe-separated helper fields.
7. Validate image filename and alt-text pairing.
8. Report derived-metric ineligibility as information or warning, not a publication error.
9. Write validation-report.json and exit non-zero for blocking errors.

5.4 Build sequence

10. Run validation.
11. Transform every valid source field into its documented JSON destination.
12. Preserve all display-relevant specification values.
13. Create filter arrays and source metadata.
14. Calculate eligible derived metrics using derived-metrics.js.
15. Exclude published = FALSE records from public machines.json.
16. Derive manufacturer, model-year and filter indexes.
17. Write deterministic JSON and build-info.json.
18. Assemble dist and run smoke tests.

5.5 Required outputs

File	Contents
dist/data/machines.json	Complete published machine records, all display specifications, helpers, source data and derived metrics.
dist/data/manufacturers.json	Sorted manufacturer index derived from published records.
dist/data/model-years.json	Unique model years sorted newest first.
dist/data/filter-options.json	Unique filter values from helper fields and published records.
dist/data/build-info.json	Build date, commit/version, source row count, published count and power-to-weight availability count.
artifacts/validation-report.json	Structured errors, warnings and derived-metric availability messages.

6. Front-end implementation

6.1 Display schema

src/js/display-schema.js will define the user-facing sections, labels, units, order, blank behaviour and delta eligibility. Every display-relevant CSV field must be represented. Helper and control fields may be omitted from ordinary specification rows while remaining available to application logic.

Section	Fields
Machine	manufacturer, machine, model_year, market, country_of_manufacture
Power and engine	rated_hp, max_hp, max_hp_with_ipm, max_torque_nm, engine_capacity_l, number_of_cylinders, engine_manufacturer, engine_stage_tier
Derived performance	power-to-weight hp/t, power-to-weight kW/t and bases
Transmission and speed	transmission, transmission_class, top_speed_kmh
Dimensions and weights	wheelbase_mm, unladen_weight_kg, maximum permissible weights at 40, 50 and 60 km/h
Hydraulics and hitch	rated and maximum hydraulic flow, max SCVs, rear and front hitch capacities
Cab and capacities	cab_suspension, fuel_capacity_l, adblue_tank_l
PTO	rear_pto_option
Source and review	source_title, source_url, last_reviewed_date, notes

6.2 Module responsibilities

Module	Responsibility
data-loader.js	Fetch complete runtime JSON using relative paths.
filters.js	Build dynamic filter controls and apply combined filters.
relationships.js	Calculate Max HP relationship bands and result ordering.
comparison.js	Render all display sections, source values and safe deltas.
power-to-weight.js	Format derived ratios, basis text, unavailable states and ratio deltas.
display-schema.js	Define every display field, labels, groups, units and delta rules.
exports.js	Include visible source and derived fields in copy, CSV and print output.
app.js	Coordinate state and application events.

6.3 Comparison and delta policy

- All display-relevant source fields are available in the detailed comparison.
- Blank fields may render as a consistent dash or be conditionally collapsed according to the display schema.
- Numeric deltas require two clean scalar numbers and a deltaEligible field definition.
- Descriptive text is displayed without extracting misleading numbers.
- Power-to-weight ratio delta is allowed only when both generated ratios are available.
- Priority specifications remain visually prominent: Max HP, Max Torque, maximum permissible weight at 40 km/h and Top Speed.

7. Testing strategy

7.1 Data and schema tests

- Every CSV header has exactly one schema mapping.
- Every display-relevant field maps to a generated JSON property and display-schema entry.
- Helper fields become filter data without duplicate specification rows.
- Missing markers become null.
- Complex descriptive fields remain strings.
- Massey Ferguson names and IDs do not reintroduce redundant MF.

7.2 Power-to-weight tests

Test	Expected result
443 HP and 12,700 kg	34.88 hp/t and 26.01 kW/t at display precision.
Missing Max HP	Both ratios null; reason power_missing_or_invalid.
Missing unladen weight	Both ratios null; reason unladen_weight_missing_or_invalid.
Weight = 0	Both ratios null; validation/derived warning.
Weight = 24639 (narrow), 25546 (wide)	Both ratios null; reason unladen_weight_not_scalar.
Weight = 8000 / 8900	Both ratios null; no automatic weight selection.
Both comparison ratios valid	Absolute and percentage ratio deltas displayed.
Only one comparison ratio valid	No ratio delta displayed.

7.3 Integration and regression tests

- Build the complete CSV and verify all generated files parse.
- Verify published record count against the source publication flags.
- Verify JSON contains every required display specification key.
- Verify manufacturer, year and filter indexes agree with machines.json.
- Verify relationship matching still uses Max HP.
- Verify comparison, copy, export and print include power-to-weight where available.
- Verify the GitHub Pages project-path build loads all JSON and assets.

8. GitHub Actions and deployment

PULL REQUEST

checkout -> setup Node -> npm ci -> tests -> CSV validation -> build -> smoke tests

MAIN BRANCH

all checks -> configure Pages -> upload dist artifact -> deploy Pages

- Protect main and require validation/test checks.
- Use feature or fix branches for data and code changes.
- Deploy only generated dist content.
- Use relative application paths.
- Keep validation and build reports as workflow artifacts.
- Do not manually edit deployed JSON.

8.1 Security model

Security position

The site is distributed through an unlisted link. The link is not authentication. The static site must not contain customer data, pricing, margins, confidential benchmarking, passwords, tokens or private keys.

9. Implementation sequence

Stage	Work	Completion proof
0. Workstation	Install Git and connect Visual Studio Code to approved GitHub.	git --version works and repository access is confirmed.
1. Foundation	Create repository structure, documentation, Copilot instructions and machines.csv.	Initial repository commit is complete.
2. Schema	Define all CSV fields and display/filter/control classifications.	All source columns are mapped exactly once.
3. Validation	Implement CSV parsing, errors, warnings and report.	Current CSV passes and deliberate errors fail clearly.
4. Data build	Generate complete JSON and indexes.	All display data is preserved and five JSON outputs exist.
5. Derived metrics	Implement power-to-weight calculation and unavailable reasons.	Eligible records calculate correctly; ambiguous weights do not.
6. Tests	Add unit, integration and regression coverage.	npm test passes.
7. Front end	Migrate existing UI to generated JSON and complete display schema.	All comparison fields and existing behaviours work.
8. Images	Implement paths, placeholders and alt text.	Representative image and no-image records work.
9. Automation	Implement Actions checks and Pages deployment.	PRs verify; main deploys.
10. Release	Complete regression checks and publish controlled URL.	Build information and product are available.

9.1 First GitHub Copilot task

	First milestone Implement schema, validation, complete CSV transformation, power-to-weight derivation, tests and JSON outputs. Do not rebuild the full interface until the complete data pipeline passes against machines.csv.
--	--

9.2 Data-pipeline definition of done

- All CSV columns are represented in schema.js.
- npm run validate passes against the prepared source.
- npm run build creates all required JSON files.
- machines.json contains every display-relevant specification field.
- Derived power-to-weight is present for eligible machines and null with a reason for ineligible machines.
- Duplicate or invalid IDs block the build.
- New manufacturers and years are discovered automatically.
- published = FALSE records are excluded from public machine output.
- Complex source values remain unchanged.
- The source CSV is never modified by the build.

10. GitHub Copilot guardrails

- Read the project documentation, this specification, copilot-instructions.md, schema.js and tests before changing code.
- Treat machines.csv as the complete source data set.
- Do not reduce generated objects to headline specifications.
- Do not add a manually entered power-to-weight CSV column.
- Do not choose a weight from a descriptive multi-option value.
- Do not infer missing specifications.
- Do not hard-code manufacturers, years or filter options.
- Keep dependencies minimal and run all checks after changes.
- Update documentation and tests when the data contract changes.

10.1 Recommended Copilot prompt addition

The complete machines.csv field set must be represented in the schema and generated JSON. All display-relevant specifications must be available in the detailed comparison.

Add derived power-to-weight metrics without modifying machines.csv:

- Power basis: max_hp
- Weight basis: unladen_weight_kg
- Formula: $(\text{max_hp} / \text{unladen_weight_kg}) * 1000$
- Also calculate kW/t using 0.745699872
- Calculate only when both inputs are clean positive scalar numbers
- Do not choose values from ranges, options or descriptive weight strings
- Include basis metadata and an unavailable reason in generated JSON
- Add unit, integration and comparison tests

11. Acceptance criteria

Area	Acceptance criterion
Data completeness	Every display-relevant CSV field is supported in generated JSON and the detailed comparison.
Maintenance	A model, year or manufacturer can be added through CSV changes without catalogue code changes.
Derived metric	Eligible machines show correct hp/t and kW/t using Max HP and unladen weight.
Ambiguous weights	No ratio is calculated from multi-option or descriptive weight values.
Comparison	Two valid ratios can be compared with safe deltas.
Validation	Identity and schema errors block deployment with understandable messages.
Existing behaviour	Relationship search, filters, comparison, copy, export and print are preserved.
Deployment	A reviewed merge to main publishes the Pages site automatically.
Security	No secrets or sensitive information are present in the static application.

Version 1.1 completion

The technical implementation is complete when the complete CSV data set powers the site, all detailed specifications are available, power-to-weight is consistently derived for eligible records, ambiguous

inputs are handled transparently, and future catalogue updates require data changes rather than application-code changes.

Appendix A. Power-to-weight copy and export labels

Field	Display label	Unit
powerToWeightHpPerTonne	Power-to-weight ratio	hp/t
powerToWeightKwPerTonne	Power-to-weight ratio	kW/t
powerBasis	Power basis	Max HP
weightBasis	Weight basis	Unladen weight
powerToWeightUnavailableReason	Availability note	Plain-language explanation

Appendix B. Data-change checklist

- ☐ Machine ID is unique and stable.
- ☐ All source fields retain intended meaning.
- ☐ Published is TRUE or FALSE.
- ☐ Max HP and unladen weight values are not altered to force a ratio.
- ☐ Complex weights remain descriptive text.
- ☐ No specification is inferred.
- ☐ Previous valid model years remain.
- ☐ Image and alt text are paired.
- ☐ Validation, tests and build pass.